软件设计模式

实验报告 1：创建型模式

学　　院 软件工程

专　　业 数据科学

姓　　名 毛乾屹

学　　号 2022083036　　班级 221

2024 年　月　日

目　录

1　实验目的 1

2　实验题目 1

3　实验步骤 1

3.1　类图设计 1

3.2　代码 2

4　运行结果 5

1 实验目的

本次实验主要学习使用创建型设计模式，通过本次实验，要求掌握创建型模式的主要应用场景和编程方法。

2 实验题目

请使用单例模式实现一个文件操作类，要求如下：

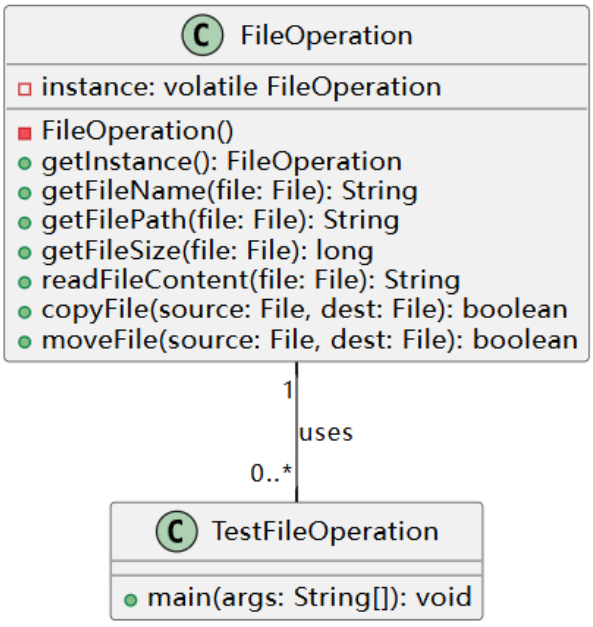
(1) 使用“懒汉式”单例

(2) 使用“双重检查锁定”来保证多线程情况下的单例；

(3) 提供最基本的文件操作功能，包括显示文件名、文件路径、文件大小等信息，读取文件内容、复制、移动文件等。

3 实验步骤

3.1 类图设计



3.2 代码

//本部分请将各个类的代码粘贴于此，并加以说明（代码中应当有适当的注释）

FileOperation.java

```
package com.yourpackage;

import java.io.*;
import java.nio.file.*;

public class FileOperation {
    // volatile 关键字确保多线程环境下的可见性
    private static volatile FileOperation instance = null;

    // 私有构造方法，防止外部实例化
    private FileOperation() {
        // 初始化操作（如果有）
    }

    // 公共的获取实例的方法，使用双重检查锁定
    public static FileOperation getInstance() {
        if (instance == null) { // 第一次检查
            synchronized (FileOperation.class) {
                if (instance == null) { // 第二次检查
                    instance = new FileOperation();
                }
            }
        }
        return instance;
    }

    // 显示文件名
    public String getFileName(File file) {
        return file.getName();
    }

    // 显示文件路径
    public String getFilePath(File file) {
        try {
            return file.getCanonicalPath();
```

```
        } catch (IOException e) {
            e.printStackTrace();
            return null;
        }
    }

    // 显示文件大小（字节）
    public long getFileSize(File file) {
        return file.length();
    }

    // 读取文件内容
    public String readFileContent(File file) {
        StringBuilder content = new StringBuilder();
        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                content.append(line).append(System.lineSeparator());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return content.toString();
    }

    // 复制文件到目标路径
    public boolean copyFile(File source, File dest) {
        try {
            Files.copy(source.toPath(), dest.toPath(),
StandardCopyOption.REPLACE_EXISTING);
            return true;
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }

    // 移动文件到目标路径
    public boolean moveFile(File source, File dest) {
        try {
```

```
            Files.move(source.toPath(), dest.toPath(),
StandardCopyOption.REPLACE_EXISTING);
            return true;
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }
}
```

TestFileOperation.java

```
package com.yourpackage;

import java.io.File;

public class TestFileOperation {
    public static void main(String[] args) {
        // 获取 FileOperation 的单例实例
        FileOperation fileOp1 = FileOperation.getInstance();
        FileOperation fileOp2 = FileOperation.getInstance();

        // 验证单例模式（两个实例应该是同一个）
        System.out.println("fileOp1 和 fileOp2 是同一个实例吗？ " + (fileOp1
== fileOp2));

        // 定义示例文件路径（请根据实际情况修改路径）
        File file = new File("src/com/yourpackage/example.txt");

        // 显示文件信息
        System.out.println("文件名: " + fileOp1.getFileName(file));
        System.out.println("文件路径: " + fileOp1.getFilePath(file));
        System.out.println("文件大小: " + fileOp1.getFileSize(file) + " 字
节");

        // 读取文件内容
        String content = fileOp1.readFileContent(file);
        System.out.println("文件内容:\n" + content);

        // 复制文件到新的位置
        // 复制后的文件名为 copy_example.txt
        File copyDest = new File("copy_example.txt");
```

```
        boolean copyResult = fileOp1.copyFile(file, copyDest);
        System.out.println("复制文件 " + (copyResult ? "成功" : "失败"));

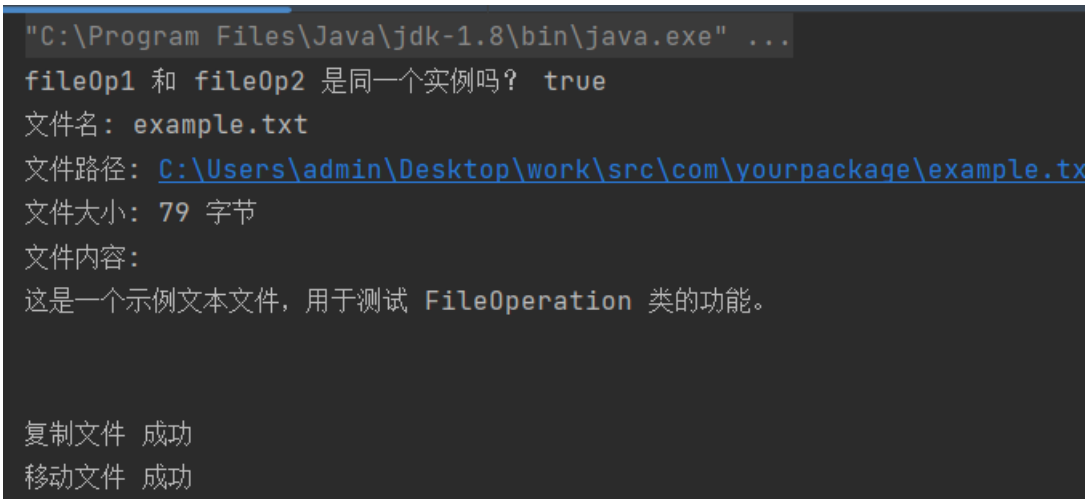
        // 移动文件到新的位置
        // 移动后的文件名为 moved_example.txt
        File moveDest = new File("moved_example.txt");
        boolean moveResult = fileOp1.moveFile(copyDest, moveDest);
        System.out.println("移动文件 " + (moveResult ? "成功" : "失败"));
    }
}
```

Example.txt

```
这是一个示例文本文件，用于测试 FileOperation 类的功能。
```

4 运行结果

//请截图说明本实验的运行结果，并对结果进行分析说明



fileOp1 和 fileOp2 是同一个实例吗？ true

解释：这行输出验证了 FileOperation 类是否真正实现了单例模式。通过比较 fileOp1 和 fileOp2 的引用，结果为 true，说明两个引用指向同一个实例，证明单例模式的正确实现。

文件名: example.txt

文件路径: C:\Users\admin\Desktop\work\src\com\yourpackage\example.txt

```
文件大小: 79 字节
```

文件名:通过 getFileName 方法获取，显示为 example.txt，与创建的文件名一致。

文件路径:通过 getFilePath 方法获取，显示为文件的绝对路径，表明文件位于指定位置。

文件大小:通过 getFileSize 方法获取，显示为 79 字节，表示文件的大小。

```
文件内容:
这是一个示例文本文件，用于测试 FileOperation 类的功能。
```

解释：通过 readFileContent 方法读取并显示了 example.txt 的内容。内容与之前在 example.txt 中编写的内容一致，表明读取功能正常。

```
复制文件 成功
移动文件 成功
```

解释：通过 copyFile 方法将 example.txt 复制为 copy_example.txt。输出成功表示复制操作顺利完成，且目标文件 copy_example.txt 已成功创建。通过 moveFile 方法将 copy_example.txt 移动为 moved_example.txt。输出成功表明移动操作已顺利完成，且目标文件 moved_example.txt 已成功创建。

5 总结

通过此次实验，成功实现了一个基于懒汉式单例模式的 FileOperation 类，并验证了其在多线程环境下的单例性。同时，实现了基本的文件操作功能，并通过测试类 TestFileOperation 验证了这些功能的正确性。运行结果显示所有功能均正常工作，程序稳定结束，证明设计和实现是正确的。

注意事项：

1. 请按照本文档的格式编写
2. 实验完成后将本文档转为 PDF 格式，以“学号-姓名-实验 1.pdf”命名
3. 将工程中的 src 目录打包成 zip 格式，以“学号-姓名-实验 1.zip”命名
4. 将上述 2 个文件发给班级的学习委员，学习委员收集后发给老师